9. Interfaces and Composition

09. Interfaces and Composition

Phase: Phase 2 — Core language depth
Estimated time: 1–2 days
Slug: `09-interfaces`

Learning objectives

- Define interfaces as sets of methods
- Accept interfaces, return concrete types in APIs
- Compose behavior via small interfaces
- Understand the empty interface and `any`

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Interfaces are how Go does polymorphism without inheritance. Small interfaces (io.Reader) are the secret to testable, composable infrastructure code.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Satisfaction is implicit: no `implements` keyword.
- Accept interfaces, return concrete types in most APIs.
- Compose interfaces by embedding (ReadWriter).
- `any` is empty interface — prefer generics or real interfaces when possible.
- Interface values hold (type, value) pairs; nil interface vs nil pointer matters.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-interfaces && cd /tmp/gw-interfaces
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import "fmt"

type Shape interface{ Area() float64 }
type Rect struct{ W, H float64 }

func (r Rect) Area() float64 { return r.W * r.H }

func TotalArea(shapes []Shape) float64 {
    var s float64
    for _, sh := range shapes {
        s += sh.Area()
    }
    return s
}

func main() {
    fmt.Println(TotalArea([]Shape{Rect{2, 3}, Rect{1, 1}}))
}
```

Run it:

```
go run .
```

Expected output:

```
7
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Define Shape + Rect; sum areas via []Shape.
2. Add Circle and pass both to TotalArea without changing TotalArea.
3. Experiment with var s Shape = (*Rect)(nil) and compare to var s Shape.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Interfaces are implicit

In Go you do **not** write `implements`. If a type has the methods, it satisfies the interface.

```
type Stringer interface {
    String() string
}

type User struct{ Name string }
func (u User) String() string { return u.Name }
```

Small interfaces win

`io.Reader`, `io.Writer`, `fmt.Stringer` — one or two methods. Write functions that take interfaces so callers can plug mocks, buffers, or network streams.

```
func CountLines(r io.Reader) (int, error) { /* ... */ }
```

Composition over inheritance

Go has no class hierarchy. Embed structs and combine interfaces instead.

```
type ReadWriter interface {
    io.Reader
    io.Writer
}
```

`any` (empty interface)

`any` is `interface{}` — every type satisfies it. Useful for containers before generics; prefer generics when you know the type set.

Failure modes to watch while coding

- Defining huge interfaces that no one can mock
- Returning interfaces from constructors unnecessarily
- Using `any` everywhere instead of real types

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Python structural typing is runtime; Go checks method sets at compile time.
- No ABC required — interfaces are satisfied implicitly.
- Fat interfaces are hard to mock; keep them tiny like io.Reader.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Standard library design: functions take io.Reader not *os.File. Copy that style in your APIs.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Interfaces and Composition” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Defining huge interfaces that no one can mock
- Returning interfaces from constructors unnecessarily
- Using `any` everywhere instead of real types

Exercises

Work in `exercises/09-interfaces/`. Read that folder's `README.md` before you code.

What you will build

Implement Rect.Area and TotalArea(shapes []Shape) float64 for the Shape interface.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/09-interfaces/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. How does a type satisfy an interface in Go?
2. Why are small interfaces preferred?
3. When is `any` acceptable?

Further reading

- https://go.dev/tour/methods
- https://go.dev/doc/effective_go#interfaces